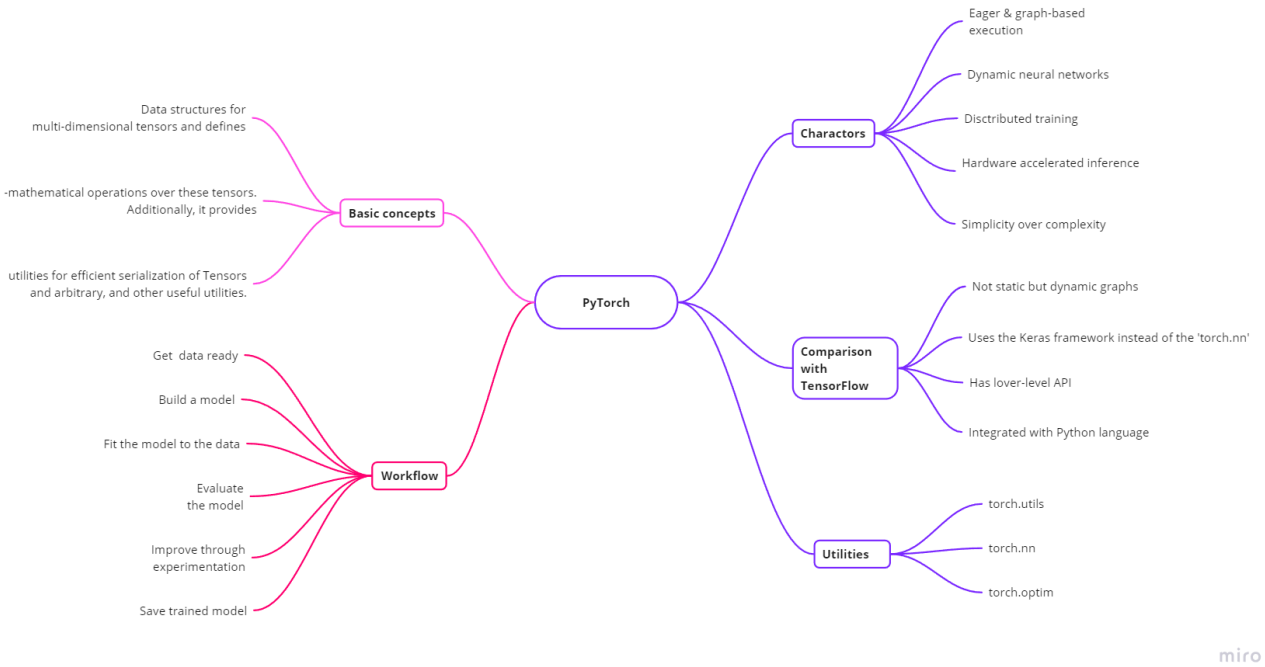
Experiment Report

Student: Denisenko Sofiia 狄苏菲
Student id: 1820249051
Course: Big Data Analysis Technology

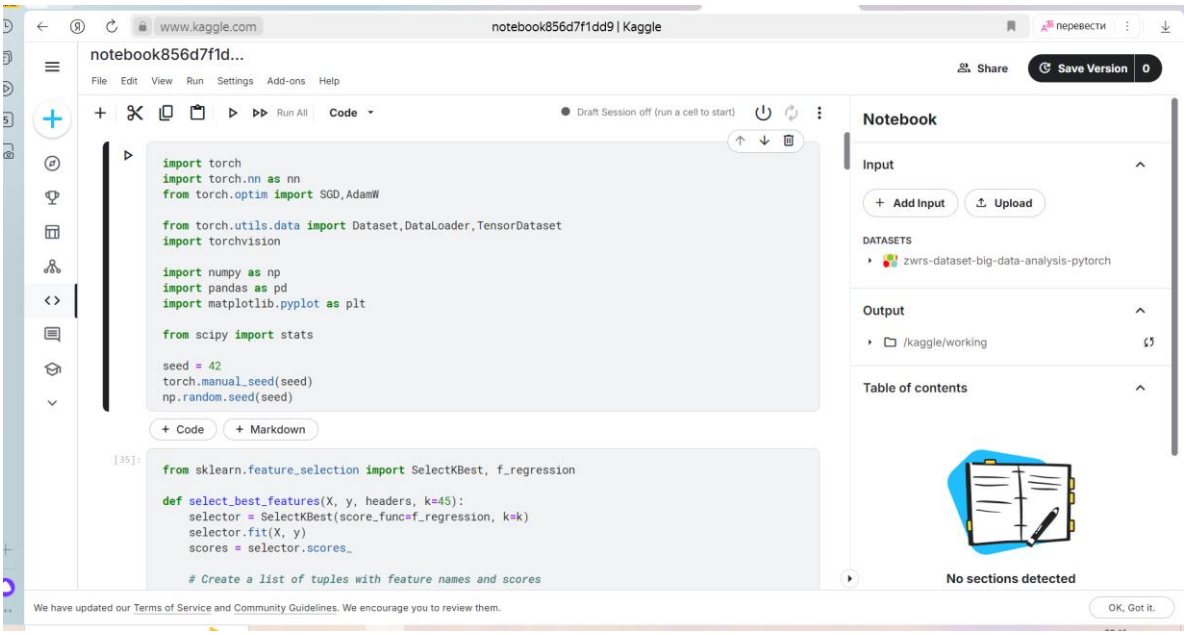
Experiment Topic: PyTorch

Mind Map PyTorch



Experiment 1

At first, I imported the .ipynb file into Kaggle and imported a dataset **covid.train.csv "ZWR's Dataset: Big Data Analysis Pytorch".**



Running the code and checking every step. Using ReLU:

```
train_data = my_dataset("/kaggle/input/zwrs-dataset-big-data-analysis-pytorch/covid.t
```

```
class my_NN(nn.Module):
    def __init__(self):
        super().__init__()
        self.Matrix1 = nn.Linear(45,16)
        self.Matrix2 = nn.Linear(16,8)
        self.Matrix3 = nn.Linear(8,1)
        self.R = nn.ReLU()

    def forward(self,x):
        x = self.R(self.Matrix1(x))
        x = self.R(self.Matrix2(x))
        x = self.Matrix3(x)

        return x.squeeze()
```

[6]:
```
f = my_NN()
f.double()
```

[6]:
```
my_NN(
  (Matrix1): Linear(in_features=45, out_features=16, bias=True)
  (Matrix2): Linear(in_features=16, out_features=8, bias=True)
  (Matrix3): Linear(in_features=8, out_features=1, bias=True)
  (R): ReLU()
)
```

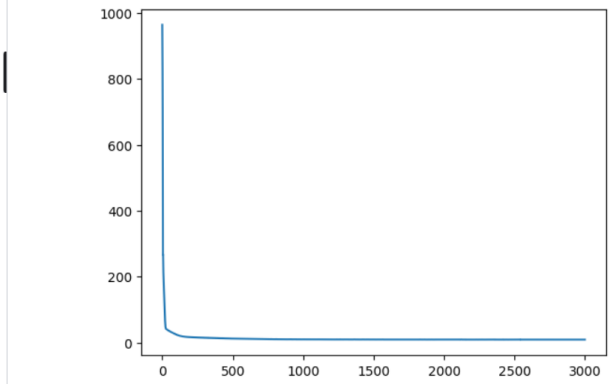
Training the model for 3000 epochs.

```
This is Epoch 2980
This is Epoch 2981
This is Epoch 2982
This is Epoch 2983
This is Epoch 2984
This is Epoch 2985
This is Epoch 2986
This is Epoch 2987
This is Epoch 2988
This is Epoch 2989
This is Epoch 2990
This is Epoch 2991
This is Epoch 2992
This is Epoch 2993
This is Epoch 2994
This is Epoch 2995
This is Epoch 2996
This is Epoch 2997
This is Epoch 2998
This is Epoch 2999
```

Making a plot where x is the number of epochs and y is a loss.

```
[11]: plt.plot(epoch_data, losses_data)
```

```
[11]: [<matplotlib.lines.Line2D at 0x7c30f9754190>]
```



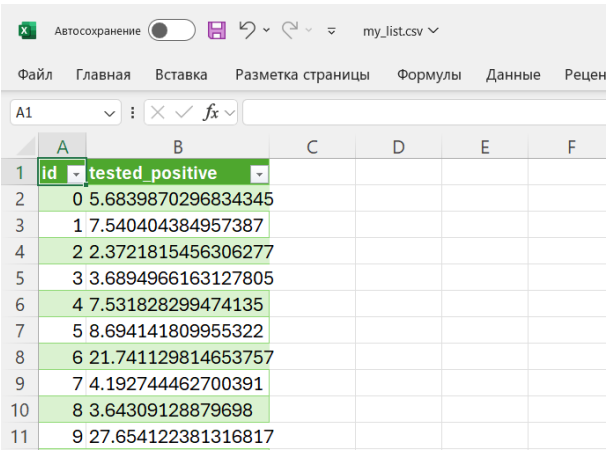
Storing the data in .csv file.

```
[13]: l = len(test_data)
      res = []
      for i in range(l):
          res.append(f(test_data[i]).item())
```

```
[14]: import csv

      with open('my_list.csv', 'w', newline='') as file:
          writer = csv.writer(file)
          writer.writerow(["id","tested_positive"])
          count = 0
          for item in res:
              writer.writerow([count,item])
              count += 1
```

Output file “**my_list.csv**”:



	A	B	C	D	E	F
1	id	tested_positive				
2	0	5.6839870296834345				
3	1	7.540404384957387				
4	2	2.3721815456306277				
5	3	3.6894966163127805				
6	4	7.531828299474135				
7	5	8.694141809955322				
8	6	21.741129814653757				
9	7	4.192744462700391				
10	8	3.64309128879698				
11	9	27.654122381316817				

```
!python /kaggle/input/zwrs-dataset-big-data-analysis-pytorch/test.py
```

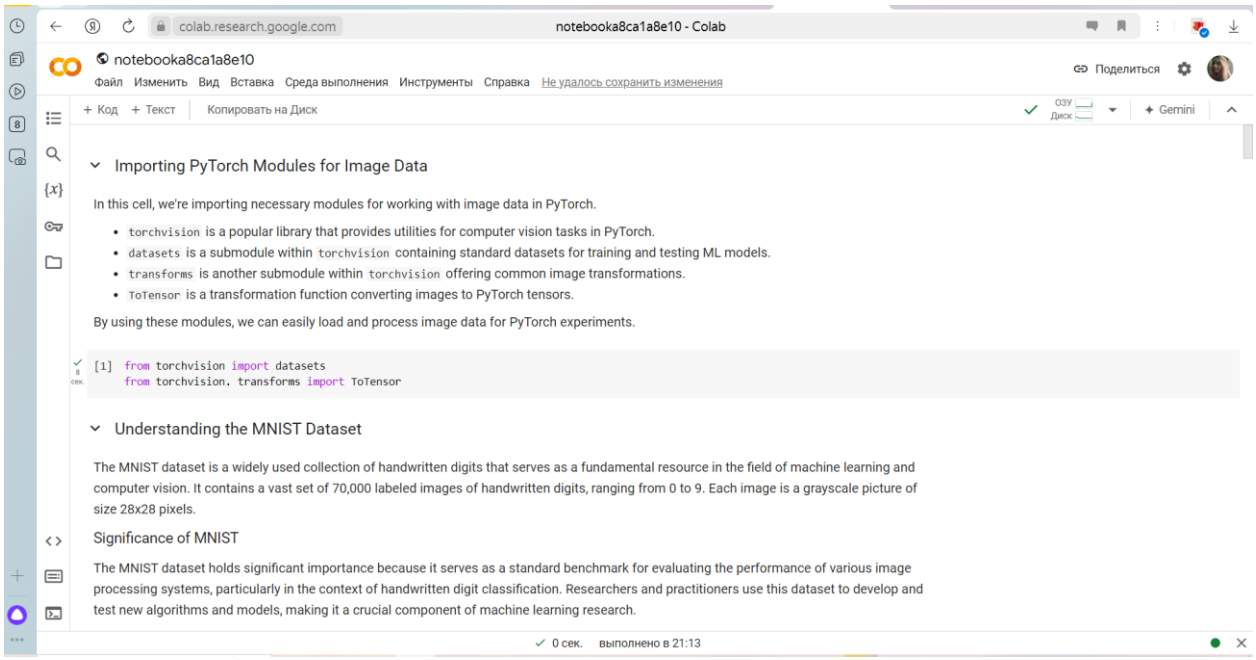
```
Random sampled data: 【1】12.839067419350044 【2】11.6573001
Mean Squared Error (MSE) Loss: 1.197516903465513
```

+ Code + Markdown

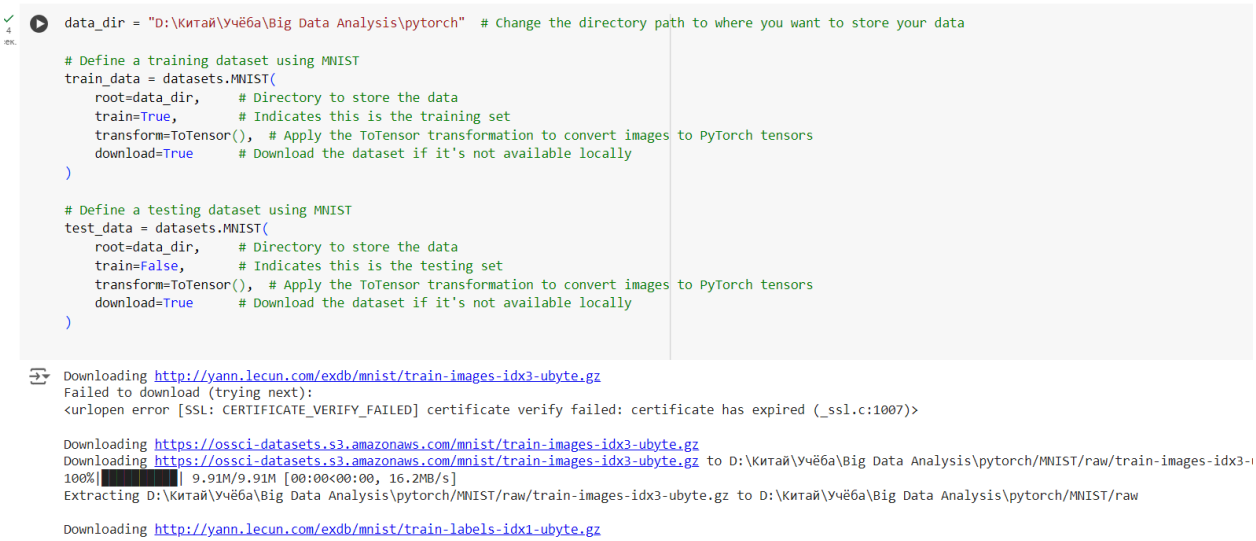
We got MSE Loss almost 1.2.

Experiment 2

I did second experiment in Google Colab, because Kaggle wasn't working for me.



Here I downloaded MNIST dataset in my local PC.



```
data_dir = "D:\Китай\Учёба\Big Data Analysis\pytorch"  # Change the directory path to where you want to store your data

# Define a training dataset using MNIST
train_data = datasets.MNIST(
    root=data_dir,     # Directory to store the data
    train=True,        # Indicates this is the training set
    transform=ToTensor(),  # Apply the ToTensor transformation to convert images to PyTorch tensors
    download=True      # Download the dataset if it's not available locally
)

# Define a testing dataset using MNIST
test_data = datasets.MNIST(
    root=data_dir,     # Directory to store the data
    train=False,       # Indicates this is the testing set
    transform=ToTensor(),  # Apply the ToTensor transformation to convert images to PyTorch tensors
    download=True      # Download the dataset if it's not available locally
)
```

```
Downloading http://yann.lecun.com/exdb/mnist/train-images-idx3-ubyte.gz
Failed to download (trying next):
<urlopen error [SSL: CERTIFICATE_VERIFY_FAILED] certificate verify failed: certificate has expired (_ssl.c:1007)>

Downloading https://ossci-datasets.s3.amazonaws.com/mnist/train-images-idx3-ubyte.gz
Downloading https://ossci-datasets.s3.amazonaws.com/mnist/train-images-idx3-ubyte.gz to D:\Китай\Учёба\Big Data Analysis\pytorch/MNIST/raw/train-images-idx3-
100%|██████████| 9.91M/9.91M [00:00<00:00, 16.2MB/s]
Extracting D:\Китай\Учёба\Big Data Analysis\pytorch/MNIST/raw/train-images-idx3-ubyte.gz to D:\Китай\Учёба\Big Data Analysis\pytorch/MNIST/raw

Downloading http://yann.lecun.com/exdb/mnist/train-labels-idx1-ubyte.gz
```

Then we divide data into training and testing sets.

```
[3] train_data
```

```
Dataset MNIST
    Number of datapoints: 60000
    Root location: D:\Китай\Учёба\Big Data Analysis\pytorch
    Split: Train
    StandardTransform
Transform: ToTensor()
```

```
[4] test_data
```

```
Dataset MNIST
    Number of datapoints: 10000
    Root location: D:\Китай\Учёба\Big Data Analysis\pytorch
    Split: Test
    StandardTransform
Transform: ToTensor()
```

```
[5] train_data.data.shape
```

```
torch.Size([60000, 28, 28])
```

```
[6] test_data.data.shape
```

```
torch.Size([10000, 28, 28])
```

```
[7] train_data.targets.size()
```

```
torch.Size([60000])
```

```
[8] train_data.targets
```

```
tensor([5, 0, 4,  ..., 5, 6, 8])
```

```
from torch.utils.data import DataLoader

# Define data loaders for training and testing datasets
loaders = {
    'train': DataLoader(
        train_data,             # Training dataset
        batch_size=100,         # Number of samples in each mini-batch
        shuffle=True,           # Shuffle the data before each epoch
        num_workers=1           # Number of processes to use for data loading
    ),
    'test': DataLoader(
        test_data,              # Testing dataset
        batch_size=100,         # Number of samples in each mini-batch
        shuffle=True,           # Shuffle the data before each epoch
        num_workers=1           # Number of processes to use for data loading
    )
}
```

```
[10] loaders
```

```
{'train': <torch.utils.data.dataloader.DataLoader at 0x78ac0c5966e0>,
 'test': <torch.utils.data.dataloader.DataLoader at 0x78ac0c594220>}
```

Training the model.

```
Train Epoch: 10 [2000/60000 (3%)]       1.50099
Train Epoch: 10 [4000/60000 (7%)]       1.51038
Train Epoch: 10 [6000/60000 (10%)]      1.58844
Train Epoch: 10 [8000/60000 (13%)]      1.5024
Train Epoch: 10 [10000/60000 (17%)]     1.48934
Train Epoch: 10 [12000/60000 (20%)]     1.51383
Train Epoch: 10 [14000/60000 (23%)]     1.49878
Train Epoch: 10 [16000/60000 (27%)]     1.51011
Train Epoch: 10 [18000/60000 (30%)]     1.54055
Train Epoch: 10 [20000/60000 (33%)]     1.53183
Train Epoch: 10 [22000/60000 (37%)]     1.53829
Train Epoch: 10 [24000/60000 (40%)]     1.52073
Train Epoch: 10 [26000/60000 (43%)]     1.50321
Train Epoch: 10 [28000/60000 (47%)]     1.5129
Train Epoch: 10 [30000/60000 (50%)]     1.52359
Train Epoch: 10 [32000/60000 (53%)]     1.548
Train Epoch: 10 [34000/60000 (57%)]     1.49798
Train Epoch: 10 [36000/60000 (60%)]     1.54152
Train Epoch: 10 [38000/60000 (63%)]     1.51329
Train Epoch: 10 [40000/60000 (67%)]     1.52978
Train Epoch: 10 [42000/60000 (70%)]     1.54883
Train Epoch: 10 [44000/60000 (73%)]     1.47937
Train Epoch: 10 [46000/60000 (77%)]     1.52408
Train Epoch: 10 [48000/60000 (80%)]     1.57441
Train Epoch: 10 [50000/60000 (83%)]     1.53318
Train Epoch: 10 [52000/60000 (87%)]     1.54136
Train Epoch: 10 [54000/60000 (90%)]     1.53641
Train Epoch: 10 [56000/60000 (93%)]     1.55794
Train Epoch: 10 [58000/60000 (97%)]     1.5103

Test set: Average loss: 0.0149, Accuracy 9754/10000 (98%
)
```

We got the 98% accuracy which is pretty high.

```
[14] device

device(type='cpu')
```

Predicting the image with a number 9. Operation success, the model is right.

```
output = model(data)
prediction = output.argmax(dim=1, keepdim=True).item()
print (f'Prediction: {prediction}')
image = data.squeeze(0).squeeze(0).cpu().numpy()
plt.imshow(image, cmap='gray')
plt.show()
```

```
Prediction: 9
<ipython-input-11-9913cbdb0d11>:33: UserWarning: Implicit dimension choice for softmax has been deprecated. Change the call to include dim=X as an argument.
  return F.softmax(x)
```